

# Distributional Reinforcement Learning | Part 2

Jonas Müller

June 29, 2026

# Outline

- 1 Recap
- 2 Scalar Linear Function Approximation
- 3 Deep Distributional Reinforcement Learning

# Recap: MDP and Return

We are working on a finite MDP:

$$(\mathcal{X}, \mathcal{A}, \xi_0, P_X, P_R), \quad \pi : \mathcal{X} \rightarrow \mathcal{P}(\mathcal{A}), \quad \gamma \in [0, 1).$$

Trajectory process:

$$\begin{aligned} X_0 &\sim \xi_0, & A_t &\sim \pi(\cdot \mid X_t), \\ R_t &\sim P_R(\cdot \mid X_t, A_t), & X_{t+1} &\sim P_X(\cdot \mid X_t, A_t), \end{aligned} \quad t \geq 0.$$

Discounted return:

$$G^\pi(x) = \sum_{t=0}^{\infty} \gamma^t R_t \quad \text{given } X_0 = x.$$

# Recap: Value function and Bellman Operator

The value function is given by the expected return:

$$V^\pi(x) = \mathbb{E}[G^\pi(x)] = \int_{\mathbb{R}} z \, d\eta_\pi(x)(z)$$

Where  $\eta_\pi(x) = \mathcal{D}(G^\pi(x))$  is the return distribution.

As we know, there is a recursive relation for the value function known as Bellman's Equation:

$$V^\pi = \mathcal{T}^\pi V^\pi.$$

Where we have the Bellman operator  $\mathcal{T}^\pi : \mathbb{R}^{\mathcal{X}} \rightarrow \mathbb{R}^{\mathcal{X}}$ :

$$(\mathcal{T}^\pi V)(x) = \mathbb{E}_\pi [R + \gamma V(X') \mid X = x].$$

## Recap: Random-Variable Bellman Equation

With  $(A, R, X')$  sampled from  $\pi(\cdot | x)P_R(\cdot | x, A)P_X(\cdot | x, A)$ , the random-variable Bellman equation is:

$$G^\pi(x) \stackrel{D}{=} R + \gamma G^\pi(X') \mid X = x.$$

Taking laws gives the return-distribution equation:

$$\eta_\pi(x) = \mathcal{D}(R + \gamma G^\pi(X') \mid X = x).$$

This is the distributional version of  $(\mathcal{T}^\pi V)(x) = \mathbb{E}_\pi[R + \gamma V(X') \mid X = x]$ .

# Recap: Distributional Bellman Operator

Define the bootstrap map

$$b_{r,\gamma}(z) = r + \gamma z, \quad r, z \in \mathbb{R},$$

and for  $Z' \sim \eta(x')$ ,

$$(b_{r,\gamma})_{\#}\eta(x') = \mathcal{D}(r + \gamma Z').$$

Distributional Bellman operator  $\mathcal{T}^{\pi} : \mathcal{P}(\mathbb{R})^{\mathcal{X}} \rightarrow \mathcal{P}(\mathbb{R})^{\mathcal{X}}$ :

$$(\mathcal{T}^{\pi}\eta)(x) = \mathbb{E}_{\pi} [(b_{R,\gamma})_{\#}\eta(X') \mid X = x].$$

Distributional Bellman equation:

$$\eta_{\pi}(x) = \mathbb{E}_{\pi} [(b_{R,\gamma})_{\#}\eta_{\pi}(X') \mid X = x], \quad \eta_{\pi} = \mathcal{T}^{\pi}\eta_{\pi}.$$

# Recap: Representations of Return Distributions

Categorical representation:

$$\eta(x) = \sum_{i=1}^m p_i^\theta(x) \delta_{\theta_i}, \quad p_i^\theta(x) \geq 0, \quad \sum_{i=1}^m p_i^\theta(x) = 1.$$

Quantile representation:

$$\eta(x) = \frac{1}{m} \sum_{i=1}^m \delta_{\theta_i(x)}, \quad \tau_i = \frac{2i-1}{2m}, \quad \theta_i(x) \approx F_{\eta(x)}^{-1}(\tau_i).$$

## Focus of this Presentation

If  $\mathcal{X}$  is large, then storing even a scalar, let alone a distribution per state is expensive. Hence, we look for suitable approximations.

# Linear Value Function Approximation | Scalar Case

For weights  $w \in \mathbb{R}^n$  approximate the value function as

$$V_w(x) = \phi(x)^\top w, \quad \nabla_w V_w(x) = \phi(x).$$

For finite  $\mathcal{X}$  and feature map  $\phi$ , stack feature rows into  $\Phi \in \mathbb{R}^{|\mathcal{X}| \times n}$ :

$$V_w = \Phi w, \quad \mathcal{F}_\phi = \{\Phi w : w \in \mathbb{R}^n\}.$$

For state-action pairs, use e.g.  $\phi : \mathcal{X} \times \mathcal{A} \rightarrow \mathbb{R}^n$  via  $Q_w(x, a) = \phi(x, a)^\top w$ .

## Idea

Instead of having a lookup table for the value of each state, we use a linear function to represent the function. **This function can be queried cheaply.**



# Linear Value Function Approximation | Scalar Case

For weights  $w \in \mathbb{R}^n$  approximate the value function as

$$V_w(x) = \phi(x)^\top w, \quad \nabla_w V_w(x) = \phi(x).$$

For finite  $\mathcal{X}$  and feature map  $\phi$ , stack feature rows into  $\Phi \in \mathbb{R}^{|\mathcal{X}| \times n}$ :

$$V_w = \Phi w, \quad \mathcal{F}_\phi = \{\Phi w : w \in \mathbb{R}^n\}.$$

For state-action pairs, use e.g.  $\phi : \mathcal{X} \times \mathcal{A} \rightarrow \mathbb{R}^n$  via  $Q_w(x, a) = \phi(x, a)^\top w$ .

## Idea

Instead of having a lookup table for the value of each state, we use a linear function to represent the function. **This function can be queried cheaply.**

**Question:** How do we get a good  $w$ ?

# Linear Approximation as Regression

Fitting a linear value function is just ordinary least squares:

$$V_w = \Phi w, \quad \min_{w \in \mathbb{R}^n} \|V^\pi - V_w\|_2.$$

which we could solve by the normal equation.

## However

In reinforcement learning, we might want to have a better approximation for some states than others.

# Weighted Norm for Linear Approximation

For some  $\xi \in \mathcal{P}(\mathcal{X})$  we define the weighted  $L_2$  norm and diagonal weight matrix

$$\|V - V'\|_{\xi,2} = \left( \sum_{x \in \mathcal{X}} \xi(x) (V(x) - V'(x))^2 \right)^{1/2}. \quad \Xi = \text{diag}(\xi(x) : x \in \mathcal{X}).$$

The problem we now want to solve is:

$$\min_{w \in \mathbb{R}^n} \|V^\pi - V_w\|_{\xi,2}.$$

# Weighted Norm for Linear Approximation

For some  $\xi \in \mathcal{P}(\mathcal{X})$  we define the weighted  $L_2$  norm and diagonal weight matrix

$$\|V - V'\|_{\xi,2} = \left( \sum_{x \in \mathcal{X}} \xi(x) (V(x) - V'(x))^2 \right)^{1/2}. \quad \Xi = \text{diag}(\xi(x) : x \in \mathcal{X}).$$

The problem we now want to solve is:

$$\min_{w \in \mathbb{R}^n} \|V^\pi - V_w\|_{\xi,2}.$$

## Proposition 9.3 | Weighted Normal Equation

If  $\xi$  has full support and the columns of  $\Phi$  are linearly independent, the unique solution is

$$w^* = (\Phi^\top \Xi \Phi)^{-1} \Phi^\top \Xi V^\pi.$$

# Projection Back into the Feature Space

We have a linear approximation now, however we can't iterate the Bellman operator because we could have

$$\mathcal{T}^\pi V_w \notin \mathcal{F}_\phi$$

Define the weighted orthogonal projection

$$(\Pi_{\phi,\xi} V)(x) = \phi(x)^\top w^*, \quad w^* \in \arg \min_w \|V - V_w\|_{\xi,2}.$$

Approximate dynamic programming therefore iterates

$$V_{k+1} = \Pi_{\phi,\xi} \mathcal{T}^\pi V_k.$$

- $\mathcal{T}^\pi$  performs the Bellman update.
- $\Pi_{\phi,\xi}$  removes components that the features cannot express.

Repeated projection can compound error (the diffusion effect).

# When Is the Projected Operator Stable?

Let  $\xi_\pi$  be a fully supported steady-state distribution under  $\pi$ .

## Projected Bellman contraction / Theorem 9.8

Since  $P^\pi$  and  $\Pi_{\phi, \xi_\pi}$  are nonexpansions in  $\|\cdot\|_{\xi_\pi, 2}$ ,

$$\|\Pi_{\phi, \xi_\pi} \mathcal{T}^\pi V - \Pi_{\phi, \xi_\pi} \mathcal{T}^\pi V'\|_{\xi_\pi, 2} \leq \gamma \|V - V'\|_{\xi_\pi, 2}.$$

Therefore there is a unique fixed point  $\hat{V}^\pi = \Pi_{\phi, \xi_\pi} \mathcal{T}^\pi \hat{V}^\pi$ , with

$$\|\hat{V}^\pi - V^\pi\|_{\xi_\pi, 2} \leq \frac{1}{\sqrt{1 - \gamma^2}} \|\Pi_{\phi, \xi_\pi} V^\pi - V^\pi\|_{\xi_\pi, 2}.$$

# Scalar Linear Approximation: Proof Chain

The scalar argument works because everything lives in a Hilbert space:

$$\mathbb{R}^{\mathcal{X}}, \quad \langle V, V' \rangle_{\xi_\pi} = \sum_{x \in \mathcal{X}} \xi_\pi(x) V(x) V'(x).$$

The feature class  $\mathcal{F}_\phi = \{\Phi w : w \in \mathbb{R}^n\}$  is a linear subspace.

- ❶  $\Pi_{\phi, \xi_\pi}$  is an orthogonal projection, hence nonexpansive.
- ❷ Stationarity of  $\xi_\pi$  makes  $P^\pi$  nonexpansive in  $\|\cdot\|_{\xi_\pi, 2}$ .
- ❸ The Bellman operator is affine with linear part  $\gamma P^\pi$ .

Therefore

$$\Pi_{\phi, \xi_\pi} \mathcal{T}^\pi \quad \text{is a } \gamma\text{-contraction.}$$

# Temporal-Difference Learning: Sampled Bellman Updates

Computing the Bellman iterates is expensive because it requires the expectation over the next state and reward. Instead, TD learning uses the sampled one-step Bellman target

$$r_k + \gamma V_w(x'_k).$$

The TD error is target minus current prediction:

$$\delta_k = r_k + \gamma V_w(x'_k) - V_w(x_k).$$

Equivalently, TD minimizes the squared TD error

$$L_k(w) = \frac{1}{2} \delta_k^2 = \frac{1}{2} (r_k + \gamma V_w(x'_k) - V_w(x_k))^2.$$

In the semi-gradient step, only  $V_w(x_k)$  is differentiated:

$$w \leftarrow w + \alpha_k \delta_k \nabla_w V_w(x_k).$$

For linear features  $V_w(x) = \phi(x)^\top w$ :

$$w \leftarrow w + \alpha_k \delta_k \phi(x_k).$$



## Section 9.4: Semi-Gradient TD

The book first writes the linear TD update as

$$w \leftarrow w + \alpha_k \underbrace{(r_k + \gamma \phi(x'_k)^\top w - \phi(x_k)^\top w)}_{\text{TD error}} \phi(x_k).$$

Then introduce a separate vector  $\tilde{w}$  in the target:

$$w \leftarrow w + \alpha_k (r_k + \gamma \phi(x'_k)^\top \tilde{w} - \phi(x_k)^\top w) \phi(x_k).$$

With fixed  $\tilde{w}$ , this is SGD on the sample loss and its solution satisfies

$$\Phi w^* = \Pi_{\phi, \xi} \mathcal{T}^\pi V_{\tilde{w}}.$$

### Book's fixed-point statement

At the fixed point  $\hat{V}^\pi = \Phi \hat{w}$  of the projected operator,

$$\mathbb{E}_\pi [(R + \gamma \phi(X')^\top \hat{w} - \phi(X)^\top \hat{w}) \phi(X)] = 0.$$

# From Value function approximation to Distributional Approximation

Want to linearly approximate probability distributions. However in general

$$\eta_{w_1}(x) + \eta_{w_2}(x) \notin \mathcal{P}(\mathbb{R})$$

and since the distribution lives in  $\mathcal{P}(\mathbb{R})$ , we can have non finite support.

## How to solve this?

Fix a finite dimensional family of distributions  $\mathcal{F}_\theta \subset \mathcal{P}(\mathbb{R})$  and then linearly approximate the parameters of the distribution. “Linear” distributional approximation means that the *parameters of a probability representation* depend linearly on features  $\phi$ .

There are therefore two approximation layers:

- 1 a finite probability representation (categorical or quantile),
- 2 linear value function approximation.

# Finite Return-Distribution Families

For a probability measure  $\eta \in \mathcal{P}(\mathbb{R})$ , we have

$$F_\eta(t) = \eta((-\infty, t]), \quad F_\eta^{-1}(\tau) = \inf\{z \in \mathbb{R} : F_\eta(z) \geq \tau\}.$$

We consider the finite dimensional approximations:

$$\text{categorical:} \quad \eta(x) = \sum_{i=1}^m p_i(x) \delta_{\theta_i}, \quad p_i(x) \geq 0, \quad \sum_{i=1}^m p_i(x) = 1,$$

$$\text{quantile:} \quad \eta(x) = \frac{1}{m} \sum_{i=1}^m \delta_{\theta_i(x)}, \quad \tau_i = \frac{2i-1}{2m}, \quad \theta_i(x) \approx F_{\eta(x)}^{-1}(\tau_i).$$

## What is state-dependent?

Categorical representations fix  $\theta_i$  and learn the probabilities  $p_i(x)$ . Quantile fixes equal probabilities  $1/m$  and learns locations  $\theta_i(x) \approx F_{\eta(x)}^{-1}(\tau_i)$ .

# Finite Return-Distribution Families

For a probability measure  $\eta \in \mathcal{P}(\mathbb{R})$ , we have

$$F_\eta(t) = \eta((-\infty, t]), \quad F_\eta^{-1}(\tau) = \inf\{z \in \mathbb{R} : F_\eta(z) \geq \tau\}.$$

We consider the finite dimensional approximations:

$$\text{categorical:} \quad \eta(x) = \sum_{i=1}^m p_i(x) \delta_{\theta_i}, \quad p_i(x) \geq 0, \quad \sum_{i=1}^m p_i(x) = 1,$$

$$\text{quantile:} \quad \eta(x) = \frac{1}{m} \sum_{i=1}^m \delta_{\theta_i(x)}, \quad \tau_i = \frac{2i-1}{2m}, \quad \theta_i(x) \approx F_{\eta(x)}^{-1}(\tau_i).$$

## What is state-dependent?

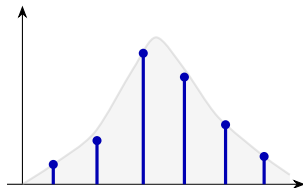
Categorical representations fix  $\theta_i$  and learn the probabilities  $p_i(x)$ . Quantile fixes equal probabilities  $1/m$  and learns locations  $\theta_i(x) \approx F_{\eta(x)}^{-1}(\tau_i)$ .

**Question:** We do not know  $F_{\eta(x)}^{-1}$  in practice.

# Categorical vs. Quantile

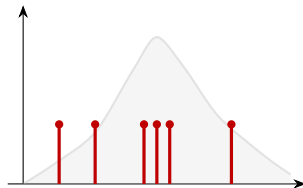
## Categorical

$$\eta(x) = \sum_{i=1}^m p_i(x) \delta_{\theta_i}$$



## Quantile

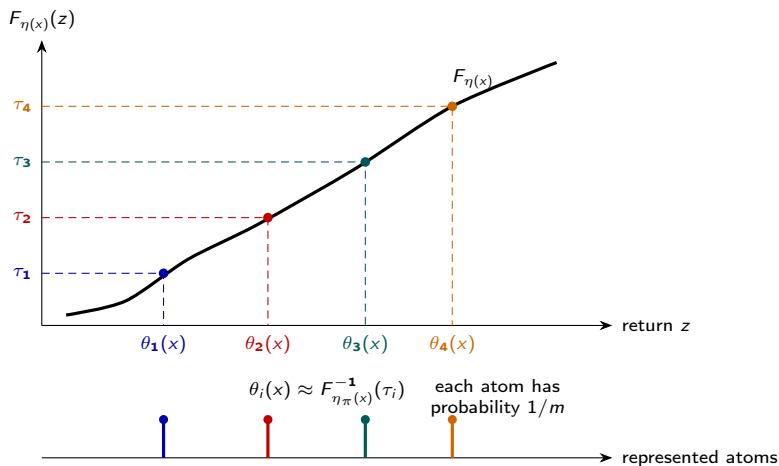
$$\eta(x) = \frac{1}{m} \sum_{i=1}^m \delta_{\theta_i(x)}$$



## Key distinction

$\tau_i$  is a fixed probability level.  $\theta_i(x)$  is the return value placed at that level.

# Quantile Levels: Input $\tau$ , Output $\theta$



Changing  $\tau_i$  asks for a different probability level; changing  $\theta_i(x)$  moves the atom and therefore changes the represented distribution.

# Distributional TD: Learning

For fixed feature map  $\phi(x) \in \mathbb{R}^d$ . A linear method has, for every atom,

$$u_i(x) = \phi(x)^\top w_i, \quad w_i \in \mathbb{R}^d.$$

Observing a transition  $(x, r, x')$  gives us an empirical target. We fit this via

$$w_i \leftarrow w_i + \alpha \varepsilon_i(x, r, x') \phi(x).$$

where  $\varepsilon_i$  is an error term. This term and the meaning of  $u_i$  is what changes.

# Linear Quantile TD: Representation and Loss

For the quantile representation, we choose

$$u_i(x) = \theta_i(x) = \phi(x)^\top w_i,$$

The represented return distribution is

$$\eta_w(x) = \frac{1}{m} \sum_{i=1}^m \delta_{\theta_i(x)} = \frac{1}{m} \sum_{i=1}^m \delta_{\phi(x)^\top w_i},$$

Then we take the quantile loss as in the tabular case

$$\rho_\tau(\Delta) = |\mathbb{1}_{\{\Delta < 0\}} - \tau| |\Delta|.$$

For one target  $z$ , atom  $i$  uses this loss on the residual

$$L_{\tau_i}(w_i) = \rho_{\tau_i}(z - \phi(x)^\top w_i),$$

whose subgradient w.r.t.  $w_i$  is  $-(\tau_i - \mathbb{1}_{\{z < \phi(x)^\top w_i\}})\phi(x)$ .



# Linear QTD: Distributional Semi-Gradient

Similar to the tabular case, for transition  $(x, a, r, x')$  we get  $m$  target samples

$$g_j = r + \gamma \phi(x')^\top w_j, \quad j = 1, \dots, m.$$

For atom  $i$ , average the quantile loss over these targets:

$$L_i(w_i) = \frac{1}{m} \sum_{j=1}^m \rho_{\tau_i}(g_j - \phi(x)^\top w_i).$$

The semi-gradient step is

$$w_i \leftarrow w_i - \alpha \nabla_{w_i} L_i(w_i) = w_i + \alpha \frac{1}{m} \sum_{j=1}^m \underbrace{(\tau_i - \mathbb{1}_{\{g_j < \phi(x)^\top w_i\}})}_{\text{quantile TD error}} \phi(x).$$

Unlike the scalar TD error, each averaged quantile error is in  $[-1, 1]$ .

# Linear Categorical TD: Prediction

We approximate the probabilities  $p_i(x; w)$  by a linear combination of features:

$$p_i(x; w) = \frac{\exp(\phi(x)^\top w_i)}{\sum_{j=1}^m \exp(\phi(x)^\top w_j)}, \quad \eta_w(x) = \sum_{i=1}^m p_i(x; w) \delta_{\theta_i}.$$

transformed into probabilities by the softmax function. Softmax is needed, direct linear coefficients need not be nonnegative or sum to one.

# Linear Categorical TD: Prediction

We approximate the probabilities  $p_i(x; w)$  by a linear combination of features:

$$p_i(x; w) = \frac{\exp(\phi(x)^\top w_i)}{\sum_{j=1}^m \exp(\phi(x)^\top w_j)}, \quad \eta_w(x) = \sum_{i=1}^m p_i(x; w) \delta_{\theta_i}.$$

transformed into probabilities by the softmax function. Softmax is needed, direct linear coefficients need not be nonnegative or sum to one.

This can be handled, later!

# Linear CTD: How to Stay on Fixed Support

For a sampled transition  $(x, r, x')$ , the Bellman update is an update of the prediction at the current state  $x$ :

$$\underbrace{\eta_w(x) = \sum_{j=1}^m p_j(x; w) \delta_{\theta_j}}_{\text{prediction at } x} \xrightarrow{\text{Observed } (r, x')} \underbrace{(b_{r, \gamma})_{\#} \eta_w(x') = \sum_{j=1}^m p_j(x'; w) \delta_{r + \gamma \theta_j}}_{\text{Empirical Bellman step for } x}.$$

Since in general  $r + \gamma \theta_j$  need not lie on the fixed support, use the categorical projection  $\Pi_c$  to project the shifted atoms back to the fixed support  $S = \{\theta_1, \dots, \theta_m\}$ . For each  $j$ , if  $\theta_\ell \leq r + \gamma \theta_j \leq \theta_{\ell+1}$ , split its mass as

$$\bar{p}_\ell += p_j(x'; w) \frac{\theta_{\ell+1} - (r + \gamma \theta_j)}{\theta_{\ell+1} - \theta_\ell}, \quad \bar{p}_{\ell+1} += p_j(x'; w) \frac{(r + \gamma \theta_j) - \theta_\ell}{\theta_{\ell+1} - \theta_\ell}.$$

This gives the projected target

$$\bar{\eta}_{x, r, x'} = \Pi_c(b_{r, \gamma})_{\#} \eta_w(x') = \sum_{i=1}^m \bar{p}_i \delta_{\theta_i}.$$

# Linear CTD: Target, Loss, and Update

To learn softmax probabilities, we use the Cross-entropy loss:

$$L(w) = - \sum_{i=1}^m \bar{p}_i \log p_i(x; w).$$

Since

$$\frac{\partial L}{\partial u_i} = p_i(x; w) - \bar{p}_i, \quad \nabla_{w_i} u_i(x) = \phi(x),$$

gradient descent gives the semi-gradient CTD update:

$$w_i \leftarrow w_i + \alpha \underbrace{(\bar{p}_i - p_i(x; w))}_{\text{CTD error}} \phi(x).$$

## Chapter 9 Objects: Signed Categorical Family

We can make our lives to develop theory easier by allowing negative measures. The signed categorical family on  $S = \{\theta_1, \dots, \theta_m\}$  is

$$\mathcal{F}_{S,m} = \left\{ \sum_{i=1}^m p_i \delta_{\theta_i} : p_i \in \mathbb{R}, \sum_{i=1}^m p_i = 1 \right\} \subseteq \mathcal{M}(\mathbb{R}).$$

Using this we can define the signed categorical return functions

$$\mathcal{F}_{S,m}^{\mathcal{X}} = \{\eta : \mathcal{X} \rightarrow \mathcal{F}_{S,m}\}.$$

## Chapter 9 Objects: Signed Categorical Family

We can make our lives to develop theory easier by allowing negative measures. The signed categorical family on  $S = \{\theta_1, \dots, \theta_m\}$  is

$$\mathcal{F}_{S,m} = \left\{ \sum_{i=1}^m p_i \delta_{\theta_i} : p_i \in \mathbb{R}, \sum_{i=1}^m p_i = 1 \right\} \subseteq \mathcal{M}(\mathbb{R}).$$

Using this we can define the signed categorical return functions

$$\mathcal{F}_{S,m}^{\mathcal{X}} = \{\eta : \mathcal{X} \rightarrow \mathcal{F}_{S,m}\}.$$

How to linearly approximate this?

## Chapter 9 Objects: Linear Signed Approximation

Linear approximation, subtracting the mean!

$$p_i(x) = \frac{1}{m} + \phi(x)^\top w_i - \frac{1}{m} \sum_{j=1}^m \phi(x)^\top w_j.$$

Then

$$\sum_{i=1}^m p_i(x) = 1, \quad p_i(x) \in \mathbb{R}.$$

The resulting family of distribution is denoted by  $\mathcal{F}_{\phi, S, m}$ .



## Chapter 9 Objects: Linear Signed Approximation

Linear approximation, subtracting the mean!

$$p_i(x) = \frac{1}{m} + \phi(x)^\top w_i - \frac{1}{m} \sum_{j=1}^m \phi(x)^\top w_j.$$

Then

$$\sum_{i=1}^m p_i(x) = 1, \quad p_i(x) \in \mathbb{R}.$$

The resulting family of distribution is denoted by  $\mathcal{F}_{\phi, S, m}$ .

We want to perform bellman updates and ensure that the result is still in  $\mathcal{F}_{\phi, S, m}$ !

## Chapter 9 Objects: Why Two Projections?

Start with a  $\eta \in \mathcal{F}_{\phi, S, m}$ . After the Bellman update,

$$\mathcal{T}^\pi \eta$$

two things can go wrong.

- 1. Distribution representation.** For a state  $x$ ,  $(\mathcal{T}^\pi \eta)(x)$  need not be supported on  $S = \{\theta_1, \dots, \theta_m\}$ . Hence we apply the categorical projection  $\Pi_c$  from before.
- 2. Linear feature representation.** Even after  $\Pi_c$ , the map  $x \mapsto (\Pi_c \mathcal{T}^\pi \eta)(x)$  need not lie in  $\mathcal{F}_{\phi, S, m}$ . Hence we need to define the feature projection by

$$\Pi_{\phi, \xi, d} \tilde{\eta} \in \arg \min_{\eta' \in \mathcal{F}_{\phi, S, m}} d(\tilde{\eta}, \eta').$$

In some suitable metric  $d$ . The final bellman update is then

$$\Pi_{\phi, \xi, d} \Pi_c \mathcal{T}^\pi \eta.$$

## Chapter 9 Objects: Cramér Geometry

We use the Cramér distance introduced prior, given by

$$\ell_2^2(\nu, \nu') = \int_{\mathbb{R}} (F_\nu(z) - F_{\nu'}(z))^2 dz$$

whenever this quantity is finite. Define

$$\mathcal{M}_{\ell_2,1}(\mathbb{R}) = \{\nu \in \mathcal{M}(\mathbb{R}) : \nu(\mathbb{R}) = 1, \ell_2(\nu, \delta_0) < \infty\}.$$

For return-distribution functions  $\eta : \mathcal{X} \rightarrow \mathcal{M}_{\ell_2,1}(\mathbb{R})$  we can again define a weighted distance

$$\ell_{\xi,2}^2(\eta, \eta') = \sum_{x \in \mathcal{X}} \xi(x) \ell_2^2(\eta(x), \eta'(x)).$$

## Chapter 9 Lemmas: Bellman and Categorical Steps

Assume rewards have finite first moments and the Markov chain under  $\pi$  has a unique stationary distribution  $\xi_\pi$ .

### Lemma 9.14: Bellman Step

For any  $\eta, \eta' \in \mathcal{M}_{\ell_2,1}(\mathbb{R})^{\mathcal{X}}$ ,

$$\ell_{\xi_\pi,2}(\mathcal{T}^\pi \eta, \mathcal{T}^\pi \eta') \leq \gamma^{1/2} \ell_{\xi_\pi,2}(\eta, \eta').$$

### Lemma 9.15: Categorical Projection

For any signed distributions  $\nu, \nu'$ ,

$$\ell_2(\Pi_c \nu, \Pi_c \nu') \leq \ell_2(\nu, \nu').$$

## Chapter 9 Lemmas: Feature Projection

The final projection is the analogue of the scalar linear projection, but applied to signed categorical distributions with the Cramér metric.

### Lemma 9.16: Linear Feature Projection

For any  $\eta, \eta' \in \mathcal{F}_{S,m}^{\mathcal{X}}$ ,

$$\ell_{\xi_{\pi},2}(\Pi_{\phi,\xi_{\pi},\ell_2}\eta, \Pi_{\phi,\xi_{\pi},\ell_2}\eta') \leq \ell_{\xi_{\pi},2}(\eta, \eta').$$

Thus each piece is either a contraction or a nonexpansion:

$\mathcal{T}^{\pi}$  contracts,  $\Pi_c, \Pi_{\psi,\xi_{\pi},\ell_2}$  do not expand.

## Chapter 9 Theorem: Doubly Projected Bellman

Composing the three preceding statements gives the actual convergence-style result for linear distributional approximation in the book.

### Theorem 9.13

Under the assumptions above, for all  $\eta, \eta' \in \mathcal{M}_{\ell_2,1}(\mathbb{R})^{\mathcal{X}}$ ,

$$\ell_{\xi_\pi,2}(\Pi_{\psi,\xi_\pi,\ell_2}\Pi_c\mathcal{T}^\pi\eta,\Pi_{\psi,\xi_\pi,\ell_2}\Pi_c\mathcal{T}^\pi\eta') \leq \gamma^{1/2}\ell_{\xi_\pi,2}(\eta,\eta').$$

Hence the projected distributional Bellman operator has a unique fixed point in  $\mathcal{F}_{\psi,S,m}$ , by Banach's fixed-point theorem.

## Chapter 9: Exact Contraction Chain

For  $\eta, \eta' \in \mathcal{M}_{\ell_2,1}(\mathbb{R})^{\mathcal{X}}$ ,

$$\begin{aligned} \ell_{\xi_\pi,2}(\Pi_{\psi,\xi_\pi,\ell_2}\Pi_c\mathcal{T}^\pi\eta, \Pi_{\psi,\xi_\pi,\ell_2}\Pi_c\mathcal{T}^\pi\eta') \\ \leq \ell_{\xi_\pi,2}(\Pi_c\mathcal{T}^\pi\eta, \Pi_c\mathcal{T}^\pi\eta') \\ \leq \ell_{\xi_\pi,2}(\mathcal{T}^\pi\eta, \mathcal{T}^\pi\eta') \\ \leq \gamma^{1/2}\ell_{\xi_\pi,2}(\eta, \eta'). \end{aligned}$$

The three lines are exactly: feature projection nonexpansion, categorical projection nonexpansion, and distributional Bellman contraction.

# From Fixed Features to Learned Features

A deep network replaces the hand-designed map  $\phi(x)$  by a learned representation. Its penultimate layer can still be read as

$$\phi_w(x) \in \mathbb{R}^n,$$

followed by linear prediction heads.

- Earlier layers learn which state distinctions matter.
- Final layers map those features to values or distribution parameters.
- Backpropagation updates both simultaneously.

Linear approximation is therefore not discarded; it remains the final layer and the conceptual model for the deep semi-gradient update.



# From DQN to Distributional DQN

Standard DQN predicts one scalar per action:

$$x \mapsto (Q_w(x, a))_{a \in \mathcal{A}}, \quad Q_w(x, a) \approx \mathbb{E}[Z(x, a)].$$

Distributional DQN predicts a return distribution per action:

$$x \mapsto (\eta_w(x, a))_{a \in \mathcal{A}}, \quad \eta_w(x, a) \approx \mathcal{D}(Z(x, a)).$$

For a transition  $(x, a, r, x')$ , the target is now a shifted future-return distribution:

$$(b_{r, \gamma})_{\#} \eta_{\tilde{w}}(x', a^*), \quad a^* \in \arg \max_{a' \in \mathcal{A}} Q_{\tilde{w}}(x', a').$$

The output has  $m$  distribution parameters per action:

$$x \mapsto \mathbb{R}^{|\mathcal{A}| \times m}.$$

- **C51**: using the categorical representation
- **QR-DQN**: using the quantile representation
- **IQN**: quantile level  $\tau$  becomes an additional input.

## C51: Prediction and Behavior

Using state action categorical representation with fixed  $\theta_1 < \dots < \theta_m$ , the network outputs  $\phi_i(x, a; w)$  and

$$p_i(x, a; w) = \frac{e^{\phi_i(x, a; w)}}{\sum_{j=1}^m e^{\phi_j(x, a; w)}}, \quad \eta_w(x, a) = \sum_{i=1}^m p_i(x, a; w) \delta_{\theta_i}.$$

The induced action value is

$$Q_w(x, a) = \sum_{i=1}^m p_i(x, a; w) \theta_i.$$

Original C51 uses  $m = 51$  evenly spaced atoms.

# C51 Learning Algorithm

Select the target-network greedy action

$$a_{\tilde{w}}(x') \in \arg \max_{a'} Q_{\tilde{w}}(x', a').$$

Construct and project the target

$$\bar{\eta}(x, a) = \Pi_c(b_{r, \gamma})_{\#} \eta_{\tilde{w}}(x', a_{\tilde{w}}(x')) = \sum_{j=1}^m \bar{p}_j \delta_{\theta_j}.$$

Then minimize the cross entropy loss  $L(w) = - \sum_{j=1}^m \bar{p}_j \log p_j(x, a; w)$  and update

$$w \leftarrow w + \alpha \sum_{i=1}^m (\bar{p}_i - p_i(x; w)) \nabla_w \theta_i(x, y; w).$$

This is pretty much the same as the linear CTD update, aside from the need to update vector valued.

# QR-DQN: Quantile Prediction

QR-DQN interprets the  $m$  outputs as locations of an equally weighted quantile distribution:

$$\eta_w(x, a) = \frac{1}{m} \sum_{i=1}^m \delta_{\theta_i(x, a; w)}, \quad Q_w(x, a) = \frac{1}{m} \sum_{i=1}^m \theta_i(x, a; w).$$

With  $a_{\tilde{w}}(x')$  greedy by this mean, the pairwise residuals are

$$\Delta_{ij} = r + \gamma \theta_j(x', a_{\tilde{w}}(x'); \tilde{w}) - \theta_i(x, a; w).$$

Every predicted quantile is compared with every target quantile.

# QR-DQN: Quantile Huber Loss

For  $\tau_i = (2i - 1)/(2m)$ , QR-DQN minimizes

$$L(w) = \frac{1}{m} \sum_{i,j=1}^m \rho_{\tau_i}^H(\Delta_{ij}), \quad \rho_{\tau}^H(u) = |\mathbb{1}_{\{u < 0\}} - \tau| H_1(u),$$

where

$$H_1(u) = \begin{cases} \frac{1}{2}u^2, & |u| \leq 1, \\ |u| - \frac{1}{2}, & |u| > 1. \end{cases}$$

## Difference to linear case

- We use the Huber loss to reduce sensitivity to outliers and have a differentiable loss.
- The loss is over all weights simultaneously, not entrywise.

# Takeaways

- ➊ Linear approximation introduces state aliasing and a weighted projection problem; the sampling distribution is central to stability.
- ➋ Semi-gradient TD tracks a projected Bellman process, and target weights make that relationship explicit.
- ➌ Linear QTD and CTD make distribution *parameters* linear in features.
- ➍ Deep methods learn the features and retain the same prediction–target–semi-gradient structure.
- ➎ C51, QR-DQN, and IQN differ mainly in how they parameterize and train the output distribution; that choice also shapes the learned representation.

# References

- Bellemare, Dabney, Rowland (2023): *Distributional Reinforcement Learning*, Chapters 9–10.
- Bellemare, Dabney, Munos (2017): A Distributional Perspective on Reinforcement Learning.
- Dabney et al. (2018): Distributional Reinforcement Learning with Quantile Regression.
- Dabney et al. (2018): Implicit Quantile Networks for Distributional Reinforcement Learning.

## Chapter 10: Why This Is Not a DQN Theorem

C51, QR-DQN, and IQN combine several ingredients:

nonlinear network + bootstrapping + greedy control + replay/target networks.

The theory is therefore much weaker than for policy evaluation.

### What Theorem 9.13 needs

A fixed policy, linear projection in a fixed metric, and a fixed operator

$$\Pi_{\psi, \xi_{\pi}, \ell_2} \Pi_c \mathcal{T}^{\pi}.$$

### What Chapter 10 uses

Nonlinear parameters and a moving control target, e.g.

$$r + \gamma Z_{\tilde{w}} \left( x', \arg \max_{a'} \mathbb{E}[Z_{\tilde{w}}(x', a')] \right).$$

So Chapter 10 inherits the projection intuition, but not a comparable general convergence theorem for deep distributional DQN.



# IQN: Make the Quantile Level an Input

C51 and QR-DQN output a fixed finite parameter vector. IQN instead models the quantile function itself:

$$(x, a, \tau) \mapsto \theta_w(x, a, \tau) \approx F_{\eta(x,a)}^{-1}(\tau).$$

The level is encoded by

$$\varphi(\tau) = (\cos(\pi \tau i))_{i=0}^{M-1},$$

transformed by a fully connected layer, and combined multiplicatively with the state features. The final action heads satisfy

$$\theta_w(x, a, \tau) = \phi_w(x, \tau)^\top w_a.$$

# IQN: Behavior and Learning

Approximate the mean with independent  $\tau_i \sim \mathcal{U}(0, 1)$ :

$$Q_w(x, a) \approx \frac{1}{m} \sum_{i=1}^m \theta_w(x, a, \tau_i).$$

For independent online levels  $\tau_i$  and target levels  $\tau'_j$ , minimize pairwise losses of

$$r + \gamma \theta_{\tilde{w}}(x', a_{\tilde{w}}(x'), \tau'_j) - \theta_w(x, a, \tau_i).$$

Sampling many level pairs reduces gradient variance. IQN approximates the whole quantile function without committing to fixed output locations.

# IQN Makes Risk Sensitivity Simple

Risk-neutral behavior samples  $\tau \sim \mathcal{U}(0, 1)$ . Lower-tail CVaR at level  $\bar{\tau}$  is

$$\text{CVaR}_{\bar{\tau}}(G_w(x, a)) = \frac{1}{\bar{\tau}} \int_0^{\bar{\tau}} \theta_w(x, a, \tau) d\tau.$$

Approximate it by changing only the sampling distribution:

$$\text{CVaR}_{\bar{\tau}}(G_w(x, a)) \approx \frac{1}{m} \sum_{i=1}^m \theta_w(x, a, \tau_i), \quad \tau_i \sim \mathcal{U}(0, \bar{\tau}).$$

Thus an implicit representation separates learning the return distribution from choosing how that distribution should guide behavior.

# How Distributional Predictions Shape Features

Split the network into learned features and final prediction heads.

- DQN trains the representation to predict one conditional mean per action.
- C51 trains it through  $m$  probability errors per action.
- QR-DQN/IQN train it through many pairwise quantile residuals.

The type and number of predictions therefore change the learned state representation. This richer auxiliary prediction problem is a leading explanation for improved control performance even when actions are ultimately selected by the mean.